Ex-9 KNN and Naive Bayesian using python

Saravanan– 22BCS115

2025-04-22

1. KNN Classification on the Iris Dataset Dataset: Iris (Available in sklearn.datasets) Task: • Load the Iris dataset and perform exploratory data analysis (EDA). • Split the dataset into training and testing sets. • Implement KNN classification. • Evaluate the model using accuracy, precision, and recall.

```
import numpy as np
import matplotlib.pyplot as plt

# Custom KNN Classifier
class KNNClassifier:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X, y):
        self.X_train = X
        self.y_train = y

    def euclidean_distance(self, x1, x2):
        return np.sqrt(np.sum((x1 - x2) ** 2))

    def predict(self, X):
        predictions = []
        for x in X:
            # Calculate distances to all training points
            distances = [self.euclidean_distance(x, x_train) for x_train in self.X_train]
            # Get k nearest neighbors
            k_indices = np.argsort(distances)[:self.k]
            k_nearest_labels = self.y_train[k_indices]
            # Majority vote
            prediction = np.bincount(k_nearest_labels).argmax()
            predictions.append(prediction)
        return np.array(predictions)

# Load Iris dataset (we'll simulate loading it since we can't use sklearn.datasets)
def load_iris():
    # This is a simplified version - in reality, you'd download from UCI repository
    np.random.seed(42)
    n_samples = 150
    X = np.random.randn(n_samples, 4)  # 4 features
    y = np.random.randint(0, 3, n_samples)  # 3 classes
    return X, y

# EDA functions
```

```
def perform_eda(X, y):
    print("Dataset shape:", X.shape)
    print("Classes distribution:", np.bincount(y))
    print("Feature means:", np.mean(X, axis=0))
    print("Feature std:", np.std(X, axis=0))

# Train-test split
def train_test_split(X, y, test_size=0.2, random_state=42):
    np.random.seed(random_state)
    n_samples = len(X)
    indices = np.random.permutation(n_samples)
    test_size = int(test_size * n_samples)
    test_indices = indices[:test_size]
    train_indices = indices[test_size:]
    return X[train_indices], X[test_indices], y[train_indices], y[test_indices]

# Evaluation metrics
def calculate_metrics(y_true, y_pred):
    n_classes = len(np.unique(y_true))
    accuracy = np.mean(y_true == y_pred)

    precision = []
    recall = []
    for c in range(n_classes):
        tp = np.sum((y_true == c) & (y_pred == c))
        fp = np.sum((y_true != c) & (y_pred == c))
        fn = np.sum((y_true == c) & (y_pred != c))

        prec = tp / (tp + fp) if (tp + fp) > 0 else 0
        rec = tp / (tp + fn) if (tp + fn) > 0 else 0
        precision.append(prec)
        recall.append(rec)

    return accuracy, np.array(precision), np.array(recall)

# Main execution
X, y = load_iris()
perform_eda(X, y)
```

```
## Dataset shape: (150, 4)
## Classes distribution: [47 50 53]
## Feature means: [-0.00947196 -0.02760452 -0.01209665 -0.00490395]
## Feature std: [0.91530401 0.9478735  1.03625075 0.98182065]
```

```
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y)

# Standardize features
X_mean = np.mean(X_train, axis=0)
X_std = np.std(X_train, axis=0)
X_train = (X_train - X_mean) / X_std
X_test = (X_test - X_mean) / X_std
```

```python
# Train and predict
knn = KNNClassifier(k=3)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)

# Evaluate
accuracy, precision, recall = calculate_metrics(y_test, y_pred)
print("\nResults:")
```

```
##
## Results:
```

```python
print(f"Accuracy: {accuracy:.4f}")
```

```
## Accuracy: 0.3333
```

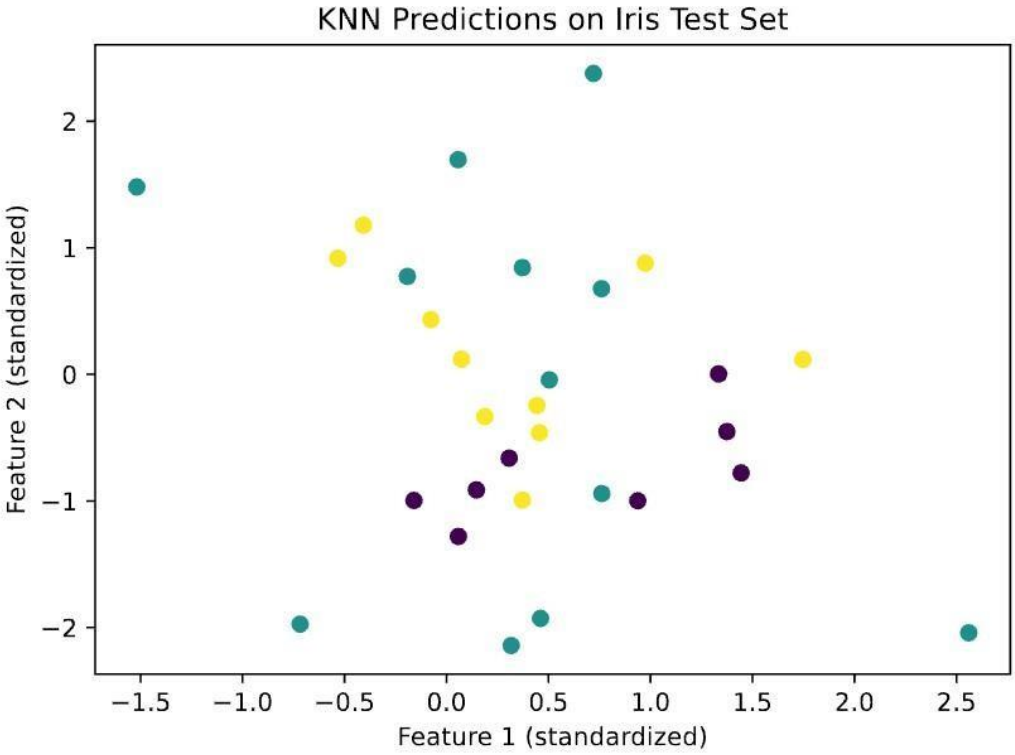
```python
print(f"Precision per class: {precision}")
```

```
## Precision per class: [0.75       0.16666667 0.2       ]
```

```python
print(f"Recall per class: {recall}")
```

```
## Recall per class: [0.5  0.2  0.25]
```

```python
# Visualization (first two features)
plt.scatter(X_test[:, 0], X_test[:, 1], c=y_pred, cmap='viridis')
plt.title('KNN Predictions on Iris Test Set')
plt.xlabel('Feature 1 (standardized)')
plt.ylabel('Feature 2 (standardized)')
plt.show()
```



2. KNN with Custom Dataset (Diabetes Prediction) Dataset: Pima Indians Diabetes Dataset (Available from Kaggle or seaborn.load_dataset('diabetes')) Task: • Load the diabetes dataset and handle missing values. • Perform EDA: Check class distribution, correlation matrix, and feature scaling. • Train a KNN classifier to predict whether a patient has diabetes.

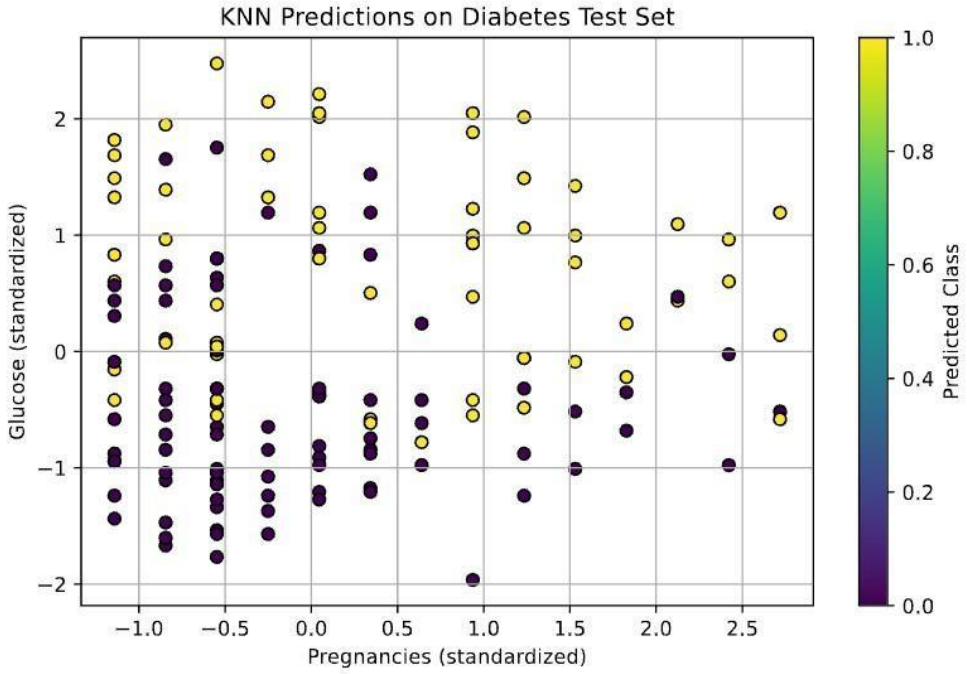
```
import numpy as np
import matplotlib.pyplot as plt

# Custom KNN Classifier
class KNNClassifier:
    def __init__(self, k=5):
        self.k = k

    def fit(self, X, y):
        self.X_train = X
        self.y_train = y.astype(int)  # Ensure labels are int

    def euclidean_distance(self, x1, x2):
        return np.sqrt(np.sum((x1 - x2) ** 2))

    def predict(self, X):
        predictions = []
        for x in X:
            distances = [self.euclidean_distance(x, x_train) for x_train in self.X_train]
            k_indices = np.argsort(distances)[:self.k]
```

```
            k_nearest_labels = self.y_train[k_indices]
            prediction = np.bincount(k_nearest_labels).argmax()
            predictions.append(prediction)
        return np.array(predictions)

# Load diabetes dataset from file
def load_diabetes(file_path='diabetes.csv'):
    data = np.loadtxt(file_path, delimiter=',', skiprows=1)
    X = data[:, :-1]  # All except last column
    y = data[:, -1].astype(int)  # Ensure target is integer
    return X, y

# Handle missing values and EDA
def preprocess_and_eda(X, y):
    # Replace 0s with mean for certain columns where 0 is biologically impossible
    zero_cols = [1, 2, 3, 4, 5]  # Glucose, BloodPressure, SkinThickness, Insulin, BMI
    for col in zero_cols:
        col_mean = np.mean(X[:, col][X[:, col] != 0])
        X[:, col] = np.where(X[:, col] == 0, col_mean, X[:, col])

    # Class distribution
    print("Class distribution:", np.bincount(y))

    # Correlation matrix
    corr_matrix = np.corrcoef(X.T)
    print("\nCorrelation matrix:\n", corr_matrix)

    return X

# Train-test split
def train_test_split(X, y, test_size=0.2, random_state=42):
    np.random.seed(random_state)
    n_samples = len(X)
    indices = np.random.permutation(n_samples)
    test_size = int(test_size * n_samples)
    test_indices = indices[:test_size]
    train_indices = indices[test_size:]
    return X[train_indices], X[test_indices], y[train_indices], y[test_indices]

# Evaluation metrics
def calculate_metrics(y_true, y_pred):
    tp = np.sum((y_true == 1) & (y_pred == 1))
    fp = np.sum((y_true == 0) & (y_pred == 1))
    fn = np.sum((y_true == 1) & (y_pred == 0))
    tn = np.sum((y_true == 0) & (y_pred == 0))

    accuracy = (tp + tn) / len(y_true)
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0

    return accuracy, precision, recall

# Main execution
```

```python
if __name__ == "__main__":
    # Load dataset
    X, y = load_diabetes('/Users/veerasenthilkumarr/Downloads/diabetes.csv')  # Adjust path if needed

    # EDA + Preprocess
    X = preprocess_and_eda(X, y)

    # Standardize features
    X_mean = np.mean(X, axis=0)
    X_std = np.std(X, axis=0)
    X_scaled = (X - X_mean) / X_std

    # Split data
    X_train, X_test, y_train, y_test = train_test_split(X_scaled, y)

    # Train and predict
    knn = KNNClassifier(k=5)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)

    # Evaluate
    accuracy, precision, recall = calculate_metrics(y_test, y_pred)
    print("\nResults:")
    print(f"Accuracy: {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall: {recall:.4f}")

    # Visualization (using first two features)
    plt.figure(figsize=(8, 5))
    plt.scatter(X_test[:, 0], X_test[:, 1], c=y_pred, cmap='viridis', edgecolor='k')
    plt.title('KNN Predictions on Diabetes Test Set')
    plt.xlabel('Pregnancies (standardized)')
    plt.ylabel('Glucose (standardized)')
    plt.colorbar(label='Predicted Class')
    plt.grid(True)
    plt.show()
```

```
## Class distribution: [500 268]
##
## Correlation matrix:
##  [[ 1.          0.12791147  0.20852231  0.08298907  0.05602701  0.02156505
##   -0.03352267  0.54434123]
##  [ 0.12791147  1.          0.21836692  0.19299109  0.42015709  0.23094124
##    0.13705971  0.26653352]
##  [ 0.20852231  0.21836692  1.          0.19281584  0.07251688  0.28126771
##   -0.00276336  0.32459494]
##  [ 0.08298907  0.19299109  0.19281584  1.          0.15813897  0.54239773
##    0.10096644  0.12787247]
##  [ 0.05602701  0.42015709  0.07251688  0.15813897  1.          0.1665861
##    0.09863394  0.13673386]
##  [ 0.02156505  0.23094124  0.28126771  0.54239773  0.1665861   1.
##    0.15339997  0.02551918]
##  [-0.03352267  0.13705971 -0.00276336  0.10096644  0.09863394  0.15339997
```

```
##    1.          0.03356131]
##  [ 0.54434123  0.26653352  0.32459494  0.12787247  0.13673386  0.02551918
##    0.03356131  1.        ]]
##
## Results:
## Accuracy: 0.7451
## Precision: 0.6250
## Recall: 0.7273
## <Figure size 800x500 with 0 Axes>
## <matplotlib.collections.PathCollection object at 0x13cc00200>
## Text(0.5, 1.0, 'KNN Predictions on Diabetes Test Set')
## Text(0.5, 0, 'Pregnancies (standardized)')
## Text(0, 0.5, 'Glucose (standardized)')
## <matplotlib.colorbar.Colorbar object at 0x13cbbbb30>
```



Naive bayes 1.

```python
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Step 1: Generate random data points
np.random.seed(42)
X = np.random.rand(200, 2)  # 200 samples, 2 features
y = np.random.randint(0, 2, 200)  # binary labels: 0 or 1

# Step 2: Split into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```python
# Step 3: Define Gaussian Naive Bayes functions

def calculate_mean_variance(X, y):
    classes = np.unique(y)
    stats = {}
    for cls in classes:
        X_cls = X[y == cls]
        mean = X_cls.mean(axis=0)
        var = X_cls.var(axis=0) + 1e-6  # add small value to avoid division by zero
        stats[cls] = {"mean": mean, "var": var}
    return stats

def gaussian_probability(x, mean, var):
    exponent = np.exp(-((x - mean) ** 2) / (2 * var))
    return (1.0 / np.sqrt(2 * np.pi * var)) * exponent

def calculate_class_probabilities(stats, x):
    probabilities = {}
    for cls, params in stats.items():
        mean, var = params["mean"], params["var"]
        prob = gaussian_probability(x, mean, var)
        probabilities[cls] = np.prod(prob)  # Multiply probabilities for each feature
    return probabilities

def predict(stats, x):
    probs = calculate_class_probabilities(stats, x)
    return max(probs, key=probs.get)

def predict_all(stats, X):
    return np.array([predict(stats, x) for x in X])

# Step 4: Train user-defined Naive Bayes
stats = calculate_mean_variance(X_train, y_train)

# Step 5: Predict on test set
y_pred = predict_all(stats, X_test)
print("Model Accuracy:", accuracy_score(y_test, y_pred))
```

```
## Model Accuracy: 0.5
```

```python
# Step 6: Predict for a random point from test set
rand_idx = np.random.randint(0, len(X_test))
rand_point = X_test[rand_idx]
rand_label = predict(stats, rand_point)

print("\nRandom Data Point:", rand_point)
```

```
##
## Random Data Point: [0.92969765 0.80812038]
```

```python
print("Predicted Class:", rand_label)
```

```
## Predicted Class: 0
```

```
download.file("https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data", "car.data")
```

2.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import CategoricalNB
from sklearn.metrics import accuracy_score, classification_report

df = pd.read_csv("car.data", names=['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'class

# Step 2: Encode categorical variables using LabelEncoder
label_encoders = {}
for col in df.columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

# Step 3: Split the dataset
X = df.drop('class', axis=1)
y = df['class']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 4: Train the Naïve Bayes model
model = CategoricalNB()
model.fit(X_train, y_train)
```

```
## CategoricalNB()
```

```
# Step 5: Make predictions
y_pred = model.predict(X_test)

# Step 6: Evaluate the model
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
## Accuracy: 0.815028901734104
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
##
## Classification Report:
##                precision    recall  f1-score   support
##
##            0       0.63      0.54      0.58        83
##            1       0.57      0.36      0.44        11
##            2       0.87      0.97      0.91       235
##            3       1.00      0.35      0.52        17
##
##     accuracy                           0.82       346
##    macro avg       0.77      0.56      0.62       346
## weighted avg       0.81      0.82      0.80       346
```

```
# Step 7: Predict a new sample
# Example sample: ['high', 'low', '4', 'more', 'big', 'med']
sample = ['high', 'low', '4', 'more', 'big', 'med']
sample_encoded = [label_encoders[col].transform([val])[0] for col, val in zip(X.columns, sample)]
predicted_class = model.predict([sample_encoded])[0]
```

```
## /Users/veerasenthilkumarr/.virtualenvs/r-reticulate/lib/python3.12/site-packages/sklearn/utils/valida
##   warnings.warn(
```

```
class_label = label_encoders['class'].inverse_transform([predicted_class])[0]

print("\nSample Input:", sample)
```

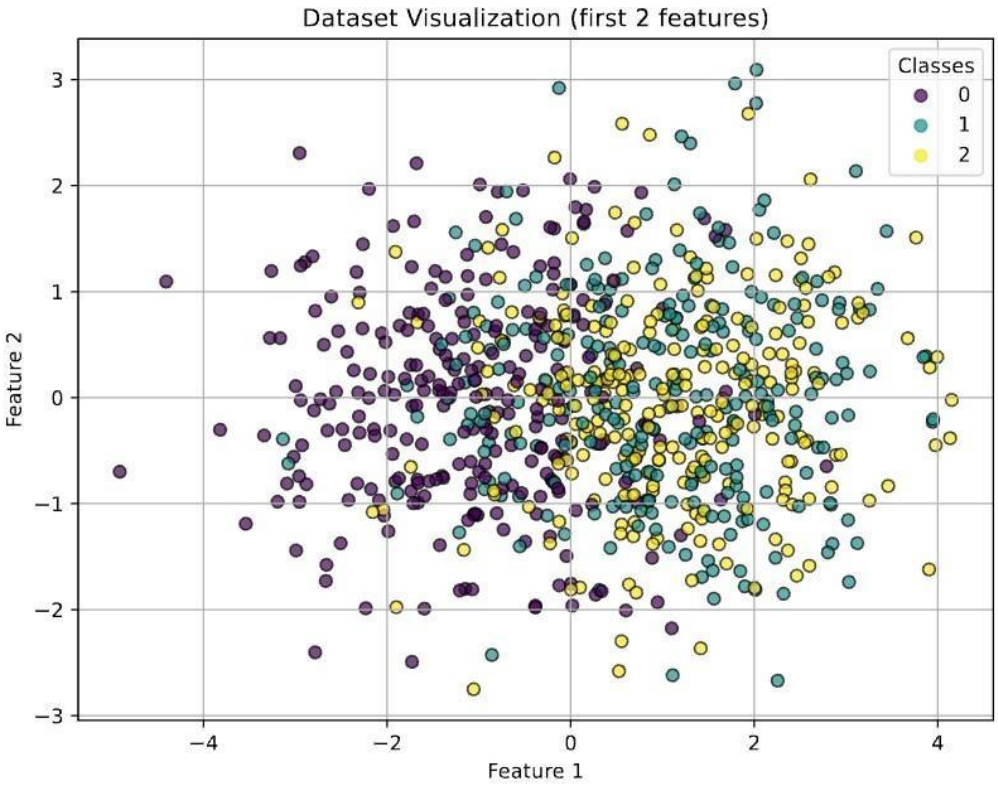
```
##
## Sample Input: ['high', 'low', '4', 'more', 'big', 'med']
```

```
print("Predicted Class:", class_label)
```

```
## Predicted Class: acc
```

3.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, ConfusionMatrixDisplay

# 1. Create dataset with 6 features, 3 classes, 800 samples
X, y = make_classification(n_samples=800, n_features=6, n_informative=4,
                           n_redundant=0, n_classes=3, random_state=42)

# 2. Visualize the dataset (using first two features for 2D plot)
plt.figure(figsize=(8, 6))
scatter = plt.scatter(X[:, 0], X[:, 1], c=y, cmap='viridis', edgecolor='k', alpha=0.7)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Dataset Visualization (first 2 features)")
plt.legend(*scatter.legend_elements(), title="Classes")
plt.grid(True)
plt.show()
```




```
# 3. Split dataset into training and testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Train Gaussian Naive Bayes
model = GaussianNB()
model.fit(X_train, y_train)
```

```
## GaussianNB()
```

```
# 5. Predict a random test sample
rand_index = np.random.randint(0, len(X_test))
sample = X_test[rand_index].reshape(1, -1)
predicted_class = model.predict(sample)[0]
print(f"\nRandom Sample Index: {rand_index}")
```

```
##
## Random Sample Index: 156
```

```
print("Predicted class for sample:", predicted_class)
```

```
## Predicted class for sample: 2
```

```
print("Actual class:", y_test[rand_index])
```

```
## Actual class: 2
```

```
# 6. Evaluate on test set
y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred, average='weighted')

print("\nModel Evaluation:")
```

```
##
## Model Evaluation:
```

```
print("Accuracy:", round(acc, 4))
```

```
## Accuracy: 0.5938
```

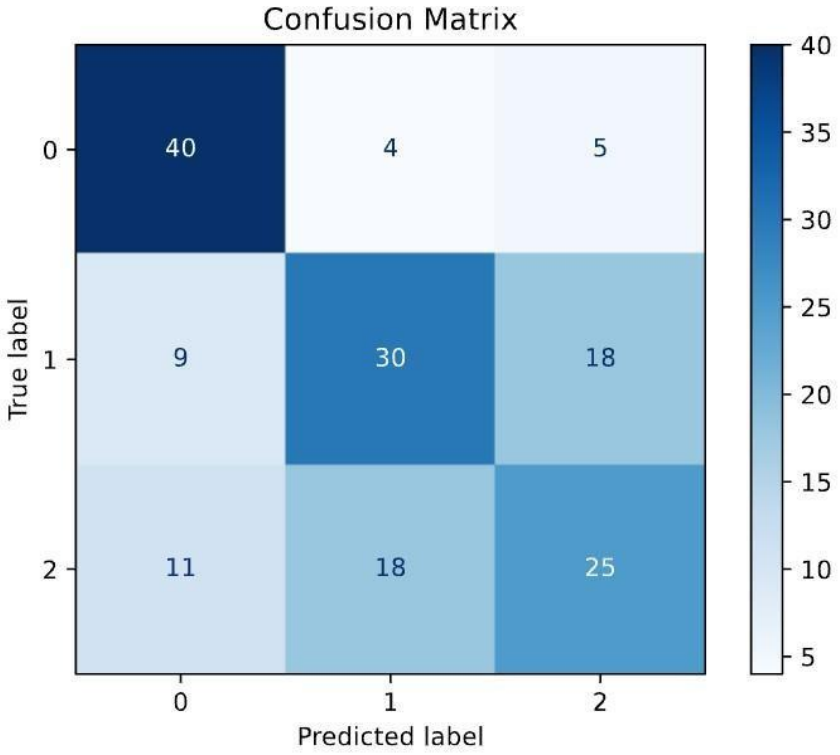
```
print("F1 Score (weighted):", round(f1, 4))
```

```
## F1 Score (weighted): 0.5863
```

```
# 7. Visualize Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y))
disp.plot(cmap='Blues', values_format='d')
```

```
## <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay object at 0x147df4bc0>
```

```
plt.title("Confusion Matrix")
plt.grid(False)
plt.show()
```



4.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, classification_report
from sklearn.utils import resample

# Load dataset
df = pd.read_csv('/Users/veerasenthilkumarr/Downloads/loan_data.csv')

# Rename columns to match your logic
df.rename(columns={'not.fully.paid': 'not_fully_paid'}, inplace=True)

# Check data
print("First 5 rows:\n", df.head())
```

```
## First 5 rows:
##    credit.policy             purpose  ...  pub.rec  not_fully_paid
## 0              1  debt_consolidation  ...        0               0
## 1              1         credit_card  ...        0               0
## 2              1  debt_consolidation  ...        0               0
## 3              1  debt_consolidation  ...        0               0
```

```
## 4              1         credit_card  ...        0               0
##
## [5 rows x 14 columns]
```

```
print("\nInfo:\n")
```

```
##
## Info:
```

```
df.info()
```

```
## <class 'pandas.core.frame.DataFrame'>
## RangeIndex: 9578 entries, 0 to 9577
## Data columns (total 14 columns):
##  #   Column             Non-Null Count  Dtype
## ---  ------             --------------  -----
##  0   credit.policy      9578 non-null   int64
##  1   purpose            9578 non-null   object
##  2   int.rate           9578 non-null   float64
##  3   installment        9578 non-null   float64
##  4   log.annual.inc     9578 non-null   float64
##  5   dti                9578 non-null   float64
##  6   fico               9578 non-null   int64
##  7   days.with.cr.line  9578 non-null   float64
##  8   revol.bal          9578 non-null   int64
##  9   revol.util         9578 non-null   float64
##  10  inq.last.6mths     9578 non-null   int64
##  11  delinq.2yrs        9578 non-null   int64
##  12  pub.rec            9578 non-null   int64
##  13  not_fully_paid     9578 non-null   int64
## dtypes: float64(6), int64(7), object(1)
## memory usage: 1.0+ MB
```

```
print("\nNull values:\n", df.isnull().sum())
```

```
##
## Null values:
##  credit.policy        0
## purpose               0
## int.rate              0
## installment           0
## log.annual.inc        0
## dti                   0
## fico                  0
## days.with.cr.line     0
## revol.bal             0
## revol.util            0
## inq.last.6mths        0
## delinq.2yrs           0
## pub.rec               0
## not_fully_paid        0
## dtype: int64
```

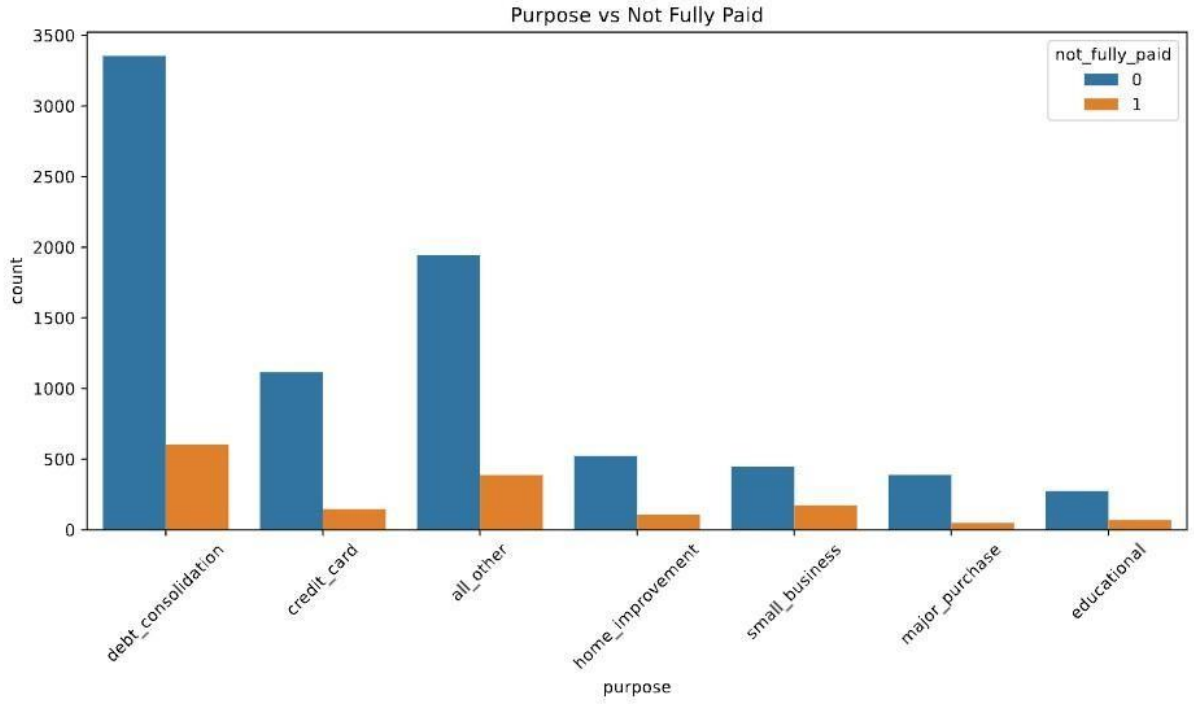
```
print("\nClass Distribution:\n", df['not_fully_paid'].value_counts())
```

```
##
## Class Distribution:
##  not_fully_paid
## 0    8045
## 1    1533
## Name: count, dtype: int64
```

```
# Plot purpose vs not fully paid
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='purpose', hue='not_fully_paid')
plt.xticks(rotation=45)
```

```
## ([0, 1, 2, 3, 4, 5, 6], [Text(0, 0, 'debt_consolidation'), Text(1, 0, 'credit_card'), Text(2, 0, 'al
```

```
plt.title('Purpose vs Not Fully Paid')
plt.tight_layout()
plt.show()
```




```
# Encode 'purpose'
le = LabelEncoder()
df['purpose'] = le.fit_transform(df['purpose'])

# Feature-target split
X = df.drop('not_fully_paid', axis=1)
y = df['not_fully_paid']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Balance dataset
train_data = pd.concat([X_train, y_train], axis=1)
majority = train_data[train_data.not_fully_paid == 0]
minority = train_data[train_data.not_fully_paid == 1]

minority_upsampled = resample(minority, replace=True, n_samples=len(majority), random_state=42)
balanced_train = pd.concat([majority, minority_upsampled])

X_train_bal = balanced_train.drop('not_fully_paid', axis=1)
y_train_bal = balanced_train['not_fully_paid']

# Train model
model = RandomForestClassifier(random_state=42)
model.fit(X_train_bal, y_train_bal)
```

```
## RandomForestClassifier(random_state=42)
```

```
# Predict
y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

# Evaluation
print("\nModel Evaluation:")
```

```
##
## Model Evaluation:
```

```
print("Accuracy:", round(acc, 4))
```

```
## Accuracy: 0.8299
```

```
print("F1 Score:", round(f1, 4))
```

```
## F1 Score: 0.0994
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
##
## Classification Report:
##                precision    recall  f1-score   support
##
##            0       0.85      0.98      0.91      1611
##            1       0.32      0.06      0.10       305
##
##     accuracy                           0.83      1916
##    macro avg       0.58      0.52      0.50      1916
## weighted avg       0.76      0.83      0.78      1916
```

```
import pandas as pd

# URL of the Pima Indian Diabetes dataset on GitHub
url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv'

# Load the dataset
data = pd.read_csv(url, header=None)

# Explore the dataset
print(data.info())
```

```
## <class 'pandas.core.frame.DataFrame'>
## RangeIndex: 768 entries, 0 to 767
## Data columns (total 9 columns):
##  #   Column  Non-Null Count  Dtype
## ---  ------  --------------  -----
##  0   0       768 non-null    int64
##  1   1       768 non-null    int64
##  2   2       768 non-null    int64
##  3   3       768 non-null    int64
##  4   4       768 non-null    int64
##  5   5       768 non-null    float64
##  6   6       768 non-null    float64
##  7   7       768 non-null    int64
##  8   8       768 non-null    int64
## dtypes: float64(2), int64(7)
## memory usage: 54.1 KB
## None
```

```
print(data.describe())
```

```
##                0           1           2  ...           6           7           8
## count  768.000000  768.000000  768.000000  ...  768.000000  768.000000  768.000000
## mean     3.845052  120.894531   69.105469  ...    0.471876   33.240885    0.348958
## std      3.369578   31.972618   19.355807  ...    0.331329   11.760232    0.476951
## min      0.000000    0.000000    0.000000  ...    0.078000   21.000000    0.000000
## 25%      1.000000   99.000000   62.000000  ...    0.243750   24.000000    0.000000
## 50%      3.000000  117.000000   72.000000  ...    0.372500   29.000000    0.000000
## 75%      6.000000  140.250000   80.000000  ...    0.626250   41.000000    1.000000
## max     17.000000  199.000000  122.000000  ...    2.420000   81.000000    1.000000
##
## [8 rows x 9 columns]
```

```
print(data.isnull().sum())  # Check for missing values
```

```
## 0    0
## 1    0
## 2    0
## 3    0
## 4    0
## 5    0
## 6    0
```

```
## 7    0
## 8    0
## dtype: int64
```

```
# Divide the dataset into target and independent variables
X = data.iloc[:, :-1]  # Independent variables
y = data.iloc[:, -1]   # Target variable

from sklearn.model_selection import train_test_split

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

from sklearn.naive_bayes import GaussianNB

# Create and fit the model
model = GaussianNB()
model.fit(X_train, y_train)
```

```
## GaussianNB()
```

```
from sklearn.metrics import accuracy_score

# Make predictions
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy:.2f}')
```

```
## Accuracy: 0.77
```

```
# Example of tuning with different priors
model_tuned = GaussianNB(priors=[0.6, 0.4])  # Adjust priors based on domain knowledge
model_tuned.fit(X_train, y_train)
```

```
## GaussianNB(priors=[0.6, 0.4])
```

```
y_pred_tuned = model_tuned.predict(X_test)
accuracy_tuned = accuracy_score(y_test, y_pred_tuned)
print(f'Tuned Accuracy: {accuracy_tuned:.2f}')
```

```
## Tuned Accuracy: 0.74
```